THENI KAMMAVAR SANGAM
COLLEGE OF ARTS AND SCIENCE (Co-Ed)
Approved by Govt. of Tamil Nadu | Affiliated to Madurai Kamaraj University
Recognized under section 2(f) of the UGC Act, 1956 | ISO 9001:2015 Certified institution
KODUVILARPATTI, THENI- 625 534.
04546 - 250882, 8925192579, www.tkscas.com, tksclgas@gmail.com

DOC-GENIE : PYTHON FUNCTION DOCSTRING GENERATOR WITH AST ANALYSIS

Submitted by

1. Aarthi K (C3S29451)
2. Abinaya K (C3S29452)
3. Anusuyadevi M (C3S29453)
4. Dharshini S (C3S29454)
5. Gayathri S (C3S29455)

Department of Computer Science

Academic Year
2025 – 2026

TABLE OF CONTENTS

S.No	Content	Page No
1	Abstract	1
2	Introduction	2
3	Objectives of the Project	3
4	Technologies Used	4
5	System Architecture	5
6	System Flowchart	6
7	Implementation Methodology	7
8	System Features	8
9	Testing and Results	9
10	Future Scope	10
11	Conclusion	11

ABSTRACT

Software documentation plays a vital role in improving the readability, maintainability, and usability of source code. However, many developers neglect writing proper documentation due to time constraints or lack of standardized formats.

The Doc-Genie Python Docstring Generator is an automated tool designed to generate documentation for Python functions automatically. The system analyzes Python code using the Abstract Syntax Tree (AST) module to identify function names, parameters, return types, and internal logic.

The generated documentation follows standard formats such as Google Style and NumPy Style docstrings. A user-friendly interface built using Gradio allows users to input Python code and instantly generate documentation.

The system also provides options to export generated documentation as PDF or TXT files. This project demonstrates how automated documentation tools can simplify the development process and improve code quality.

INTRODUCTION

In modern software development, documentation is an essential part of the coding process. Documentation helps developers understand how functions work, what parameters they require, and what values they return.

Python supports documentation through docstrings, which are special strings written inside functions, classes, and modules. Docstrings explain the purpose and usage of code components.

Despite their importance, many developers skip writing docstrings because it requires extra time and effort. As a result, large projects often contain poorly documented code.

The Doc-Genie project addresses this issue by automatically generating docstrings for Python functions. The system uses the AST module to analyze Python code and extract important information such as function name, parameters, and return values.

The generated documentation follows widely used formats such as Google Style and NumPy Style, making the code easier to understand and maintain.

OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To automatically generate Python docstrings from source code
- To reduce manual effort required for documentation
- To support standard docstring formats (Google and NumPy)
- To provide a simple and interactive user interface
- To allow exporting documentation as PDF and TXT files
- To improve code readability and maintainability

TECHNOLOGIES USED

Python

Python is the main programming language used to implement the project and develop the system logic.

AST (Abstract Syntax Tree)

The AST module is used to analyze Python code and extract information about functions, parameters, and return types.

Gradio

Gradio is used to build a web-based user interface where users can input Python code and generate docstrings interactively.

ReportLab

ReportLab is used to generate PDF files containing the generated documentation.

Autopep8 and Black

These libraries help format Python code according to standard style guidelines.

Google Colab

Google Colab provides a cloud-based environment for developing and running the application.

SYSTEM ARCHITECTURE

The system architecture consists of several modules that work together to analyze Python code and generate documentation.

User Interface Module

Users input Python functions through the Gradio web interface.

Code Analysis Module

The AST parser analyzes the structure of the Python code and extracts function details such as name, parameters, and return types.

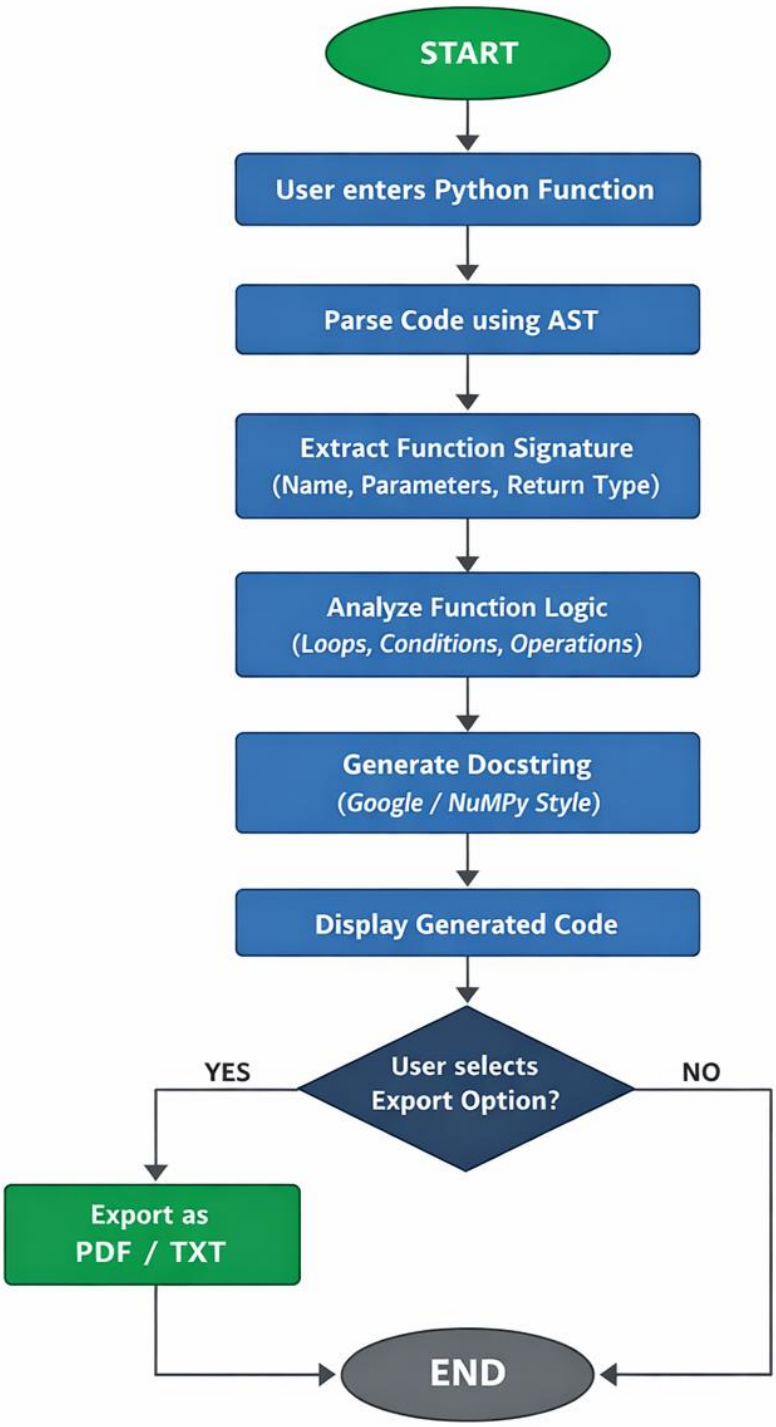
Docstring Generation Module

This module generates documentation based on the extracted information and formats it according to the selected docstring style.

Export Module

Allows users to download generated documentation as PDF or TXT files.

SYSTEM FLOWCHART



IMPLEMENTATION METHODOLOGY

Environment Setup

Required libraries such as Gradio, ReportLab, Autopep8, and Black are installed.

Code Parsing

The Python AST parser reads the code and identifies functions and parameters.

Logic Detection

The system detects logical elements such as loops, conditions, arithmetic operations, and function calls.

Docstring Generation

Extracted information is formatted into structured documentation using Google or NumPy style.

Interface Development

A user-friendly interface is built using the Gradio framework.

Export Feature

The ReportLab library generates downloadable documentation files.

SYSTEM FEATURES

The system provides the following features:

- Automatic Python docstring generation
- Support for Google and NumPy documentation styles
- Code structure analysis using AST
- Detection of loops and conditional logic
- User-friendly web interface
- Export documentation as PDF and TXT files

TESTING AND RESULTS

The system was tested using several Python functions to evaluate the accuracy of the generated documentation.

Test Case 1

Input Function

```
def add_numbers(a: int, b: int) -> int:
    return a + b
```

Generated Docstring

```
"""
Add numbers function.

Args:
    a (int): Description of a
    b (int): Description of b

Returns:
    int: Description of return value
"""
```

Test Case 2

Input Function

```
def calculate_total(price, quantity):
    total = price * quantity
    return total
```

The system correctly detected parameters and generated appropriate documentation.

The testing results confirm that the tool works effectively for generating Python docstrings automatically.

FUTURE SCOPE

The project can be further improved with the following features:

- Support for analyzing multiple functions in a file
- Automatic documentation for entire Python modules
- Integration with GitHub repositories
- AI-based parameter descriptions
- Support for additional documentation formats such as Sphinx

CONCLUSION

The Doc-Genie Python Docstring Generator simplifies the process of writing documentation for Python code. By analyzing functions using the AST module, the system automatically generates structured docstrings.

The Gradio interface makes the tool easy to use, while export features allow users to save documentation for future reference.

This project demonstrates how automation can improve software development by reducing manual effort and encouraging better documentation practices.